

Comparative Performance Analysis of Parallelization Frameworks for K-Means : Implementation on CPU and GPU

Emna Sellami^{1,2} and Yosr Slama^{1,2}

¹LIPSIC Research Lab, FST, University of Tunis El Manar, Tunis, Tunisia

²Faculty of Sciences of Tunis, University of Tunis El Manar, Tunis, Tunisia

emna.sellami@etudiant-fst.utm.tn, yosr.slama@fst.utm.tn

Abstract. Although the k-means algorithm is a popular technique for unsupervised clustering, when applied to large-scale datasets, its computational cost becomes a significant drawback. Despite the fact that the parallelization provides a practical way to increase scalability and performance, the effects of various parallel computing frameworks are still unknown. Five parallel k-means implementations are compared in this paper: a GPU-based implementation using CUDA and four CPU-based frameworks (Multiprocessing, Ray, Dask, and Parsl). To guarantee uniformity and equitable comparison, every implementation is created from the ground up. A medical imaging dataset associated with diabetic retinopathy is used for experiments. The GPU implementation outperforms sequential execution by $42.1\times$, according to the results. Parsl outperforms all CPU frameworks with a speedup $3.7\times$ at 8 threads. However, parallel efficiency drastically declines from 87% to 27% for Parsl. This highlights important trade-offs when choosing the right parallelization frameworks for large-scale clustering tasks.

Keywords: CPU, Dask, GPU, K-means clustering, Multiprocessing, Parallel computing, Parallelization frameworks, Parsl, Performance evaluation, Ray, Unsupervised classification

1 Introduction

One of the core methods in machine learning is unsupervised clustering, which makes it possible to find hidden structures in unlabeled data. Because of its ease of use and efficiency, k-means [5] stands out among these methods as one of the most popular algorithms. Its method starts with a random initialization of k centroids, where k is the number of predefined clusters. The algorithm updates centroid positions by computing the mean of associated points after assigning each data point to the closest centroid based on Euclidean distance at each iteration. The cycle of assignment updates continues until convergence, which is when the centroids stop changing significantly or the maximum number of iterations is reached. K-means is simple, but it has a big problem: it can cost a lot of time to run. For large datasets, the time it takes to run increases quickly

with the number of data points, feature dimensions, and iterations needed for convergence. This makes sequential implementation not useful for many real-world applications. Eventually, parallelization has emerged as a promising approach to accelerate k-means and improve its scalability. In this paper, we look into and compare different ways to make the k-means algorithm run in parallel in section 2. Our main goal is to measure how different parallel computing frameworks affect performance, including execution time, speedup, and parallel efficiency. As explained in section 3, both the sequential and parallel versions are built from scratch making comparisons more trustworthy. We evaluate five parallelization approaches—four CPU-based frameworks (Multiprocessing, Ray, Dask, and Parsl) and a GPU-based implementation using CUDA—described in section 4. Experimental evaluations on a diabetic retinopathy medical imaging dataset with the experimental setup detailed in section 5 demonstrate performance trade-offs between computational gains and resource utilization in section 6, providing practical guidance for selecting appropriate frameworks for large-scale clustering tasks. Conclusions are presented in section 7.

2 Related Work

Numerous researchers have investigated the parallelization of the k-means algorithm. The general principle involves partitioning data among multiple computing entities that each perform local clustering, while a master process aggregates intermediate results, recalculates global centroids, and verifies convergence criteria.

Bhimani et al. [4] compared k-means parallelization across OpenMP (shared memory, 16 cores, 128 GB RAM), MPI (distributed memory, 1-10 nodes), and CUDA (NVIDIA Tesla K20m GPU, 2496 cores) architectures using image datasets (300×300 to 1164×1200 pixels). Their key contribution was a parallel initialization strategy that simultaneously evaluated multiple random centroid sets on separate threads for 5 iterations using $\sqrt{n}$ points. All three implementations achieved $30\text{-}35\times$ speedups, with OpenMP excelling for smaller images ($30.93\times$) and CUDA for larger images ($30.26\times$). The parallel initialization provided an additional $1.5\text{-}2.5\times$ speedup by reducing convergence iterations.

Another study by Alguliyev et al. [1] proposed a parallel batch k-means algorithm that divides datasets into equal-sized batches, applies k-means in parallel to each batch, then merges and re-clusters centroids to produce final centers. Experiments on six UCI datasets (434,874 to 2,458,285 instances) using an Intel Xeon X5670 CPU (24 cores, 24 GB RAM) demonstrated speedups ranging from 109% to 19,818%, with $198\times$ achieved on the US Census dataset (2,458,285 instances) using batch size 5,000. However, quality degradation was observed on very large datasets with high cluster counts, and the approach inherited k-means' sensitivity to initial centroid selection.

In concurrent work Shang et al. [10] compared Simple K-Means and Parallel K-Means using a deterministic centroid initialization method ($\lfloor n/k \rfloor$) and client-server architecture. Experiments on educational datasets (10,000 students, 5,000

employees) showed Parallel K-Means achieved 45-50% execution time reduction and converged in exactly 3 iterations versus 7-15 for Simple K-Means. The key advantage was cluster quality and stability: Parallel K-Means generated perfectly reproducible results across all 10 executions, eliminating variability from random initialization.

3 Algorithms

In this section, we present the k-means algorithm in both its sequential and parallel versions. Our main objective is to implement parallel k-means using different parallel computing frameworks, then conduct a rigorous comparative performance analysis between these approaches and the sequential reference version.

3.1 Sequential K-Means

Our sequential k-means implementation establishes the baseline for comparative evaluation. The algorithm follows an iterative process: it begins with deterministic initialization of centroids to zero, then alternates between computing Euclidean distances from each data point to all centroids (Equation 1), assigning points to their nearest centroid, and updating centroids as the mean of their assigned points. The process continues until convergence is achieved (when centroid displacement falls below a tolerance threshold) or the maximum number of iterations is reached.

$$d(x, c) = \sqrt{\sum_{j=1}^d (x_j - c_j)^2} \quad (1)$$

where $x = (x_1, x_2, \dots, x_d)$ is a data point, $c = (c_1, c_2, \dots, c_d)$ is a centroid, and d is the number of dimensions.

The time complexity is $\mathcal{O}(n \times k \times d \times T)$, where n is the number of data points, k is the number of clusters, d is the number of dimensions, and T is the number of iterations.

3.2 Parallel K-Means

Having established the sequential version, we now describe our parallelization strategy. The approach employs multiple computing entities called workers, which can be physical entities (processors or cores) or logical entities (processes or threads). The parallelization principle relies on data partitioning among workers, enabling simultaneous processing on separate data partitions.

The process begins with the main worker initializing centroids to zero and partitioning the dataset into equal (or near-equal) partitions for distribution among workers.

For each worker i , the number of assigned points is calculated as:

$$p_i = \left\lfloor \frac{n + n_workers - i}{n_workers} \right\rfloor \quad (2)$$

where n represents the total number of points and $n_workers$ is the number of workers. This formula ensures balanced distribution, with some workers receiving one additional point when n is not perfectly divisible by $n_workers$.

Each worker independently processes its data partition by: (1) computing Euclidean distances between its points and all centroids, (2) assigning each point to the nearest cluster, and (3) calculating partial sums of point coordinates for each cluster. A synchronization barrier ensures the main worker receives all partial results before proceeding. The main worker then aggregates these partial results to compute new centroids by averaging point positions within each cluster. Convergence is verified by comparing centroid variations against a predefined threshold. If convergence is not achieved, a new iteration begins with updated centroids; otherwise, the algorithm terminates and returns the final clusters and centroids.

Algorithm 1 summarizes the parallel K-Means process.

Algorithm 1 Parallel K-Means

Require: Data X , number of clusters k , number of workers $n_workers$, maximum iterations T

Ensure: Final clusters and centroids C

```

1: /* Initialization */
2: Initialize centroids  $C$  to zero
3: Partition  $X$  into  $n\_workers$  equal partitions
4: Distribute centroids and partitions to workers
5: repeat
6:   /* Local step executed by each worker in parallel */
7:   for each worker  $w$  in parallel do
8:     Compute Euclidean distances between partition points and centroids  $C$ 
9:     Assign each point to nearest cluster
10:    Calculate partial sums of points for each cluster
11:    Send partial results to main worker
12:  end for
13:  /* Synchronization */
14:  Wait for all workers to complete (synchronization barrier)
15:  /* Aggregation and update */
16:  Aggregate all received partial sums and counts
17:  Compute new centroids  $C_{new}$  by averaging points in each cluster
18:  Check convergence
19: until convergence or maximum iterations reached
20: return Final clusters and centroids  $C$ 

```

The time complexity is:

$$\mathcal{O}\left(T \times \left(\left\lceil \frac{n \times k \times d}{n_workers} \right\rceil + Sync\right)\right)$$

where n is the number of data points, k is the number of clusters, d is the number of dimensions, T is the number of iterations, $n_workers$ is the number of workers, and $Sync$ represents synchronization and communication overhead.

4 Frameworks

We experiment with several parallelization frameworks to quantify the impact of each technology on the algorithm’s overall efficiency:

Multiprocessing [9]: Python’s built-in library for process-based parallelism, utilizing multiple CPU cores through independent processes with separate memory spaces, well-suited for CPU-bound tasks.

Ray [7]: A distributed computing framework designed for scalable parallel and distributed Python applications, featuring efficient task scheduling and shared memory optimization for machine learning workloads.

Dask [8]: A flexible parallel computing library that scales Python workflows from single machines to clusters, providing familiar APIs similar to NumPy and Pandas for easy integration.

Parsl [3]: A parallel scripting library for Python that enables execution of parallel workflows across different computing resources, from laptops to supercomputers, with minimal code modification.

CUDA: NVIDIA’s [6] parallel computing platform and programming model for GPU acceleration, enabling massive parallelism through thousands of concurrent threads for compute-intensive operations.

5 Experimental Setup

To evaluate the effectiveness of our sequential and parallel k-means implementations, we present our experimental methodology. We describe the dataset used, the hardware environment, and the evaluation metrics employed to validate correctness and measure performance improvements.

5.1 Dataset

Our experiments utilize a medical image dataset of diabetic retinopathy¹, an ocular complication of diabetes characterized by progressive retinal blood vessel damage. The dataset contains fundus images capturing retinal structures and vascular abnormalities. Each image was preprocessed and vectorized into a 196,608-dimensional feature representation. Although originally labeled, we removed class labels to make the dataset suitable for our unsupervised approach.

¹ Dataset available at: <https://www.kaggle.com/datasets/tanlikesmath/diabetic-retinopathy-resized>

We varied the dataset size from 500 to 2,000 images to observe the impact of data volume on performance.

5.2 Hardware Environment

All experiments were conducted on a virtual machine with the following specifications: QEMU Virtual CPU version 2.5+ (x86_64 architecture) with 12 threads and 32 GB RAM for CPU-based implementations, and an NVIDIA Tesla T4 GPU with 15 GB memory and approximately 5,120 CUDA cores (Google Colab) for GPU-based implementations.

5.3 Algorithm Evaluation

Our evaluation focuses on two main objectives: ensuring correctness through comparison with scikit-learn’s reference implementation, and quantifying performance improvements from parallelization. For accuracy validation, we compare our implementations against scikit-learn using the following metrics: Calinski-Harabasz Index [2] (higher values indicate better clustering), Davies-Bouldin Index [12] (lower values indicate better clustering), Silhouette Coefficient [11] (values close to 1 indicate good clustering), and direct centroid comparison to verify identical or nearly identical cluster centers. For performance evaluation, we measure execution time using Python’s *time* library. Each algorithm (sequential or parallel) is executed 10 times per configuration to account for environmental fluctuations, and the average execution time is calculated for objective comparison between implementations. We vary the number of observations to analyze how computation time scales with dataset size.

6 Experimental Results

This section presents the experimental results obtained from evaluating our k-means implementations. Our evaluation follows four stages: (1) determining the optimal number of clusters for our dataset, (2) validating implementation correctness against scikit-learn’s reference implementation, (3) evaluating parallelization framework performance across varying data sizes and thread counts, and (4) synthesizing findings to identify optimal configurations.

6.1 Optimal Number of Clusters

Before performing unsupervised clustering, we determined the optimal number of clusters using the elbow method based on within-cluster inertia [5]. We fixed a sample of 1,000 images, each represented by 196,608 features, and executed K-means for k values ranging from 2 to 10 clusters.

The inertia curve revealed a characteristic elbow between $k = 3$ and $k = 4$. To confirm the optimal choice, we evaluated configurations $k = 3$ and $k = 4$

using three internal validation metrics: Calinski-Harabasz Index, Davies-Bouldin Index, and Silhouette Coefficient. Table 1 summarizes the results.

Table 1: Comparison of evaluation metrics

Number of clusters (k)	Calinski-Harabasz	Davies-Bouldin	Silhouette
$k = 3$	457.6695	1.4639	0.2235
$k = 4$	354.7144	1.8961	0.1569

The $k = 3$ configuration demonstrates superior performance across all metrics: higher Calinski-Harabasz Index (457.67 vs. 354.71), lower Davies-Bouldin Index (1.46 vs. 1.90), and higher Silhouette Coefficient (0.22 vs. 0.16). Consequently, we selected $k = 3$ clusters for all subsequent experiments.

6.2 Accuracy Validation

With $k = 3$ established as optimal, we compared three implementations: our sequential version, our parallel version, and scikit-learn’s reference implementation using Calinski-Harabasz Index, Davies-Bouldin Index, and Silhouette Coefficient.

All three implementations produce identical metrics and cluster distributions (207, 349, and 444 images), confirming the validity of our implementations. Additionally, the mean Euclidean distance between paired centroids from our implementations and scikit-learn’s is 0.00013 ($\approx 1.3 \times 10^{-4}$), demonstrating near-perfect convergence and validating the precision of our algorithms in the feature space.

6.3 Performance Evaluation

Following validation, we evaluated the performance of our parallel implementations across different frameworks. All implementations (sequential, parallel, and GPU) converged in the same number of iterations for each dataset size, ensuring consistent final results: cluster distributions, centroids, and evaluation metrics. The number of iterations to convergence varied by dataset size: 7 iterations for 500 images, 9 for 1,000 images, 17 for 1,500 images, and 16 for 2,000 images.

Execution Time Analysis Table 2 presents execution times in seconds for varying dataset sizes (500 to 2,000 images with 196,608 features each), number of threads (2 to 12), and frameworks.

CPU frameworks demonstrate consistent patterns across all dataset sizes (Table 2, Figures 1-2). Execution times decrease from 2 to 6 threads before stabilizing or degrading due to thread management overhead. Parsl consistently

Table 2: Execution time in seconds by dataset size, number of threads, and frameworks

Dataset size	Sequential	Number of threads	Ray	Dask	Multiprocessing	Parsl	GPU
500	18.307	2	14.397	14.722	12.788	11.219	1.669
		4	10.071	10.928	8.428	7.116	
		6	8.351	9.854	7.877	5.852	
		8	8.094	9.679	8.294	6.08	
		10	9.105	9.664	8.23	6.875	
		12	9.258	10.196	9.024	6.628	
1000	46.669	2	31.561	36.559	33.101	28.924	2.571
		4	19.748	25.219	21.452	17.099	
		6	18.148	23.185	18.923	13.839	
		8	17.652	22.908	19.075	12.965	
		10	16.968	22.571	16.549	12.748	
		12	16.755	22.417	17.591	13.482	
1500	139.134	2	86.725	90.343	80.188	79.358	3.305
		4	50.048	63.954	50.451	45.178	
		6	39.623	58.957	49.126	36.828	
		8	36.939	54.572	50.514	39.946	
		10	37.701	59.675	51.132	36.819	
		12	38.632	61.969	53.43	44.09	
2000	172.599	2	104.509	112.072	103.134	99.202	5.872
		4	63.611	76.514	64.217	57.901	
		6	58.864	69.166	60.599	49.327	
		8	62.785	72.402	66.356	46.370	
		10	64.005	78.127	64.443	48.762	
		12	64.846	77.788	64.115	53.201	

Fig. 1: Execution time evolution for 2,000-instance dataset

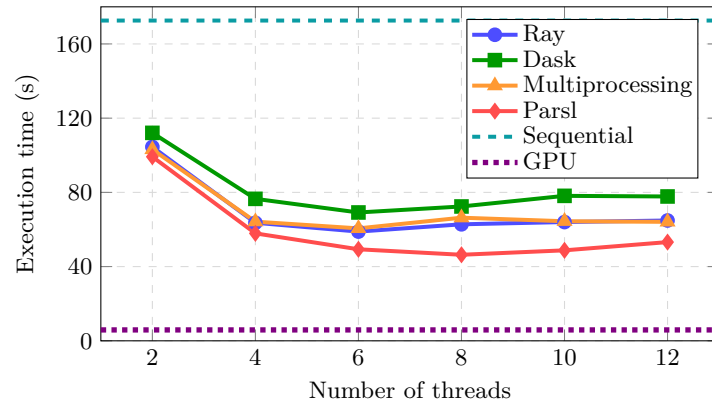
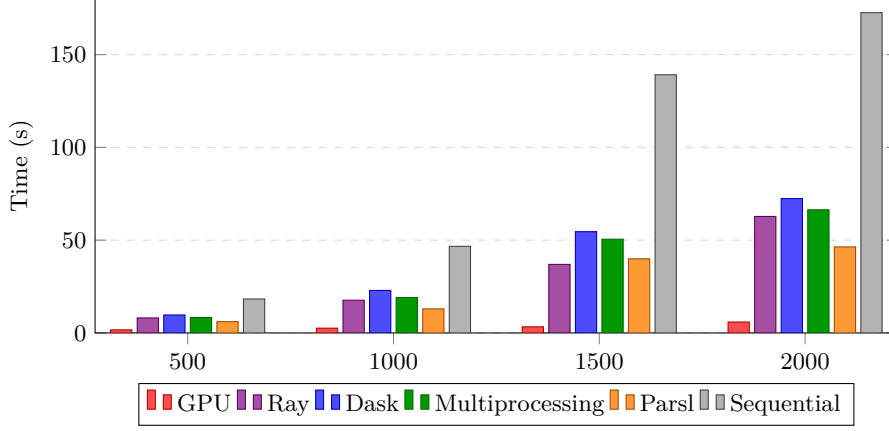


Fig. 2: Performance comparison at 8 threads across dataset sizes



achieves best CPU performance (5.852s to 46.370s for 500-2000 images at optimal thread counts), followed by Ray and Multiprocessing, with Dask showing highest times. GPU implementation substantially outperforms all CPU approaches (1.669s to 5.872s), with performance gaps widening as data size increases. Sequential execution (18.307s to 172.599s) confirms parallelization benefits of 3-4 $\times$ for CPU implementations.

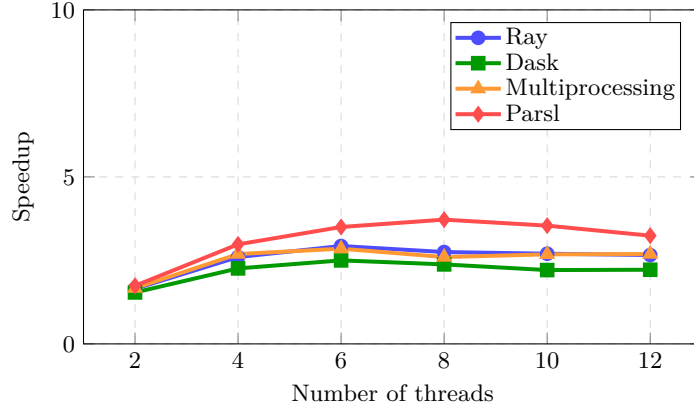
Speedup Analysis Speedup S measures the performance gain of the parallel version relative to the sequential version, calculated as:

$$S = \frac{T_S}{T_P} \quad (3)$$

where T_S represents sequential execution time and T_P represents parallel execution time.

Figure 3 illustrates speedup evolution across frameworks for different numbers of threads for the 2,000-instance dataset. We selected this largest dataset as it most clearly reveals framework characteristics with greater computational workload. CPU frameworks show relatively similar curves, with speedup increasing up to 8 threads before stabilizing or slightly declining. Parsl achieves the highest speedup at approximately 3.7 $\times$ with 8 threads, followed by Ray and Multiprocessing (2.6-2.9 $\times$) and Dask (2.5 $\times$). GPU implementation exhibits dramatically different behavior with approximately 29.4 $\times$ speedup, far surpassing all tested CPU approaches and demonstrating the power of massively parallel GPU computing for k-means.

Fig. 3: Speedup evolution with different frameworks as a function of thread count for 2,000-instance dataset



Efficiency Analysis Parallel efficiency measures the degree of effective utilization of computational resources, calculated as:

$$\text{Efficiency} = \frac{S}{n_workers} \quad (4)$$

where $n_workers$ represents the number of threads used.

Fig. 4: Efficiency evolution (%) as a function of thread count with different frameworks for 2,000-instance dataset

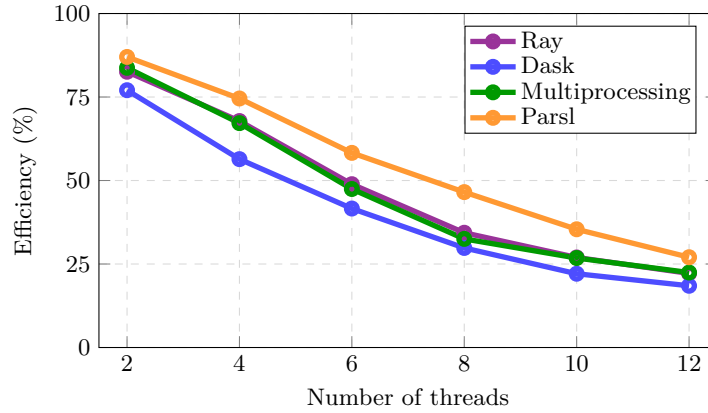


Figure 4 illustrates parallel efficiency evolution across frameworks as a function of thread count for the 2,000-instance dataset. All frameworks exhibit a

common trend: progressive efficiency decrease as thread count increases, With only 2 threads, all frameworks achieve high efficiency above 75%, with Parsl leading at 87.0%, followed by Multiprocessing (83.5%), Ray (82.5%), and Dask (77.0%). This strong initial efficiency reflects relatively low parallelization overhead compared to distributed computation gains.

As thread count increases, degradation intensifies differently across frameworks. Parsl maintains the best efficiency throughout, sustaining 74.5% at 4 threads, 58.3% at 6 threads, and 46.5% at 8 threads, before dropping to 27.0% at 12 threads. Multiprocessing follows a similar trajectory with 67.0% at 4 threads and 32.5% at 8 threads. Ray and Dask show more pronounced degradation, reaching 34.4% and 29.8% respectively at 8 threads, then 22.2% and 18.5% at 12 threads. This figure confirms that beyond 6-8 threads, parallel efficiency becomes modest for all frameworks, with less than one-third of resources effectively utilized.

7 Discussion and Conclusion

Our comparative analysis reveals key insights for optimizing parallel k-means clustering. Parsl emerges as the most performant CPU solution, consistently achieving the best speedups and efficiencies across all configurations. Ray and Multiprocessing constitute solid alternatives, while Dask presents the most modest results. GPU implementation far surpasses all CPU approaches with $42.1\times$ speedup, confirming its superiority for massively parallelizable computations, though cost and complexity may constrain certain applications.

Regarding optimal thread count, 6-8 threads represents the best compromise for CPU frameworks on medium to large datasets. Beyond this threshold, speedup gains become marginal while parallel efficiency degrades significantly. However, optimal choice depends on the primary objective: prioritizing efficiency (60-70%) favors 2-4 threads, while minimizing execution time favors 6-8 threads before overhead dominates.

Our from-scratch implementation enabled precise measurement and fair comparison, while zero-initialization ensured deterministic results. The high dimensional diabetic retinopathy dataset (196,608 features) provided a realistic testbed for medical imaging applications.

Future work should investigate: (1) adaptive thread selection strategies that dynamically adjust parallelism based on dataset characteristics, (2) extending analysis to other clustering algorithms (DBSCAN, hierarchical clustering) to assess framework ranking generalizability, and (3) evaluating distributed multi-node implementations for large-scale datasets exceeding single-machine capacity.

References

- [1] Rasim M. Alguliyev, Ramiz M. Aliguliyev, and Lyudmila V. Sukhostat. "Parallel batch k-means for Big data clustering". In: *Computers Industrial*

- Engineering* 152 (2021), p. 107023. DOI: <https://doi.org/10.1016/j.cie.2020.107023>.
- [2] Ilham Firman Ashari et al. “Analysis of elbow, silhouette, Davies-Bouldin, Calinski-Harabasz, and rand-index evaluation on k-means algorithm for classifying flood-affected areas in Jakarta”. In: *Journal of Applied Informatics and Computing* 7.1 (2023), pp. 95–103. DOI: <https://doi.org/10.30871/jaic.v7i1.4947>.
 - [3] Yadu Babuji et al. “Parsl: Pervasive parallel programming in python”. In: *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing*. 2019, pp. 25–36. DOI: <https://doi.org/10.1145/3307681.3325400>.
 - [4] Janki Bhimani, Miriam Leeser, and Ningfang Mi. “Accelerating K-Means clustering with parallel implementations and GPU computing”. In: *High Performance Extreme Computing Conference (HPEC)*. Nov. 2015, pp. 1–6. DOI: 10.1109/HPEC.2015.7322467.
 - [5] Mengyao Cui. “Introduction to the k-means clustering algorithm based on the elbow method”. In: *Accounting, Auditing and Finance* 1.1 (2020), pp. 5–8. DOI: 10.23977/accaf.2020.010102.
 - [6] *GeForce Official Site: Graphics Cards, Gaming Laptops & More*. [Online; accessed 2025-12-18]. URL: <https://www.nvidia.com/en-me/geforce/>.
 - [7] Philipp Moritz et al. “Ray: A distributed framework for emerging {AI} applications”. In: *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 2018, pp. 561–577.
 - [8] Tim Peters. *Parallel Python with Dask: Perform distributed computing, concurrent programming and manage large dataset*. GitforGits, 2023. ISBN: 978-8119177653.
 - [9] Python. *multiprocessing — Process-based parallelism*. URL: <https://docs.python.org/3/library/multiprocessing.html>.
 - [10] Rui Shang et al. “Analysis of simple K-mean and parallel K-mean clustering for software products and organizational performance using education sector dataset”. In: *Scientific Programming* 2021.1 (2021), p. 9988318. DOI: <https://doi.org/10.1155/2021/9988318>.
 - [11] Xu Wang and Yusheng Xu. “An improved index for clustering validation based on Silhouette index and Calinski-Harabasz index”. In: *IOP Conference Series: Materials Science and Engineering*. Vol. 569. 5. IOP Publishing. 2019, p. 052024. DOI: 10.1088/1757-899X/569/5/052024.
 - [12] Yahan Yu et al. “Research of Fault Feature Extraction and Analysis Method Based on Aeroengine Fault Data”. In: *2020 Chinese Automation Congress (CAC)*. IEEE. 2020, pp. 2960–2965. DOI: 10.1109/CAC51589.2020.9327519.